İSTİNYE ÜNİVERSİTESİ İSTANBUL

Capstone Project Final Report

Submitted to

Faculty of Engineering and Natural Sciences

Department of COMPUTER ENGINEERING

In partial fulfillment of the requirements for the degree

BACHELOR of SCIENCE in COMPUTER ENGINEERING

LSTM-Based Host Intrusion Detection on Linux System Call Sequences

Student: Emirhan Müftüoğlu | Muhammet İsmail Kurtuluş

ID Number: 200722102 | 210901045

Supervisor(s): Sedat Akleylek

Istanbul

May 2026

STUDENT DECLARATION

By submitting this report, as partial fulfillment of the requirements of the Capstone Project, the students promise on penalty of failure of the course that

- They have given credit to and declared (by citation), any work that is not their own (e.g. parts of the report that is copied/pasted from the Internet, design or construction performed by another person, etc.),
- They have not received unpermitted aid for the project design, construction, report or presentation,
- They have not falsely assigned credit for work to another student in the group, and not take credit for work done by another student in the group.

Abstract

This capstone project investigates whether anomalous Linux process behavior can be detected from system call sequences learned under normal operating conditions. The study is conducted as a single-scenario case study on the LID-DS 2021 dataset using the PHP_CWE-434 scenario. An end-to-end offline host-based intrusion detection pipeline is implemented, including trace parsing, preprocessing, thread-based window generation, sequence encoding, and trace-level decision making. Three methods are compared: a classical STIDE baseline, an LSTM syscall-only next-token predictor, and an LSTM context-aware next-token predictor that combines syscall, process, and return-status context. Window scores are aggregated into trace scores using top-10% mean pooling, and operating thresholds are derived from validation-normal quantiles (q95 as the primary operating point and q90 as sensitivity analysis). At q95, the LSTM Syscall model achieves the best balanced performance (Precision 0.9469, Recall 0.9386, F1 0.9427, FPR 0.0079, AUC 0.9962), while LSTM Context reaches perfect recall but with higher false positives. The findings show that deep sequence models can improve recall without sacrificing low false-positive behavior when calibrated carefully, yet the prototype remains an offline, single-scenario study and should not be over-generalized to production-scale real-time deployment.

Keywords: Host-Based Intrusion Detection; System Calls; LSTM; Anomaly Detection; LID-DS 2021.

Table of Contents

List of Tables

List of Figures

List of Abbreviations

IDS	Intrusion Detection System
HIDS	Host-Based Intrusion Detection System
LSTM	Long Short-Term Memory
STIDE	Sequence Time-Delay Embedding
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
FPR	False Positive Rate
TP	True Positive
TN	True Negative
FN	False Negative
CSV	Comma-Separated Values

Section 1:

Introduction

1. Introduction

1.1 Background and Motivation

Host-based intrusion detection systems (HIDS) analyze internal host behavior to detect suspicious activity that may bypass perimeter defenses. Linux system calls provide fine-grained behavioral signals because every user-space process eventually interacts with the kernel through syscall interfaces. This makes syscall traces a strong candidate for anomaly detection studies, especially for control-flow deviations and attack-related execution patterns (Forrest et al., 1996; Hofmeyr et al., 1998; Warrender et al., 1999).

1.2 Problem Statement

Many anomaly-based HIDS studies report high detection rates, but practical performance is often sensitive to data splitting policy, threshold selection, and false-positive control. The project problem addressed here is to build and evaluate a trace-level anomaly detection pipeline that is reproducible and transparent for a modern Linux syscall dataset scenario, while preventing overclaims beyond available evidence.

1.3 Research Aim and Objectives

Aim: to evaluate whether anomalous Linux process behavior can be reliably detected from normal syscall patterns in an offline host-based setting. Objectives are: (i) build an end-to-end data processing and evaluation pipeline; (ii) compare STIDE with LSTM-based predictors; (iii) compare syscall-only and context-aware representations; (iv) calibrate thresholds with validation-normal quantiles; and (v) analyze error patterns and limitations.

1.4 Research Questions

RQ1: Can anomalous Linux process behavior be reliably detected from normal patterns learned from system call sequences? RQ2: How does a classical STIDE baseline compare with LSTM-based next-token prediction under trace-disjoint evaluation? RQ3: Does adding process and return-status context improve detection trade-offs at trace level?

1.5 Contributions

The project contributes: (1) a reproducible offline pipeline from raw traces to trace-level decisions; (2) explicit trace-disjoint split checks; (3) a controlled comparison between STIDE, LSTM Syscall, and LSTM Context; (4) top-10% mean pooling for trace aggregation; and (5) report-ready final tables, figures, and a local Streamlit demonstration component for analyst-facing inspection.

1.6 Scope and Limitations

This report is a single-scenario case study (LID-DS 2021, PHP_CWE-434) and an offline prototype. It does not claim real-time deployment readiness. Generalization to other scenarios, organizations, or threat landscapes is not guaranteed without additional multi-scenario and operational validation.

1.7 Structure of the Report

Section 2 reviews related literature and identifies the project gap. Section 3 details the data, models, and evaluation design. Section 4 presents quantitative findings with sensitivity and error analysis. Section 5 discusses implications, limitations, and future work.

Section 2:

Literature Review

2. Literature Review

2.1 Host-Based Intrusion Detection and System Call Signals

HIDS research has evolved from rule-based monitoring to statistical and machine learning approaches that model normal host behavior and detect deviation. System calls remain a central source because they encode low-level program behavior with strong semantic relevance for host compromise analysis (Bridges et al., 2019; Hofmeyr et al., 1998).

2.2 Classical Sequence-Based Detection (n-grams, STIDE, and Related)

Foundational works modeled short syscall subsequences and mismatch behavior to detect anomalies (Forrest et al., 1996; Warrender et al., 1999). STIDE-style approaches remain competitive when tuned carefully, and operational behavior is strongly tied to n-gram length and threshold calibration (Tan & Maxion, 2002). Evasion studies such as mimicry attacks also showed that feature design and context can materially affect robustness (Wagner & Soto, 2002).

2.3 Statistical and Machine Learning Extensions

Beyond fixed n-gram matching, studies explored automata, hidden Markov models, and richer statistical features (Sekar et al., 2001; Liao & Vemuri, 2003; Hu et al., 2009; Hoang et al., 2009; Ye et al., 2001; Ye et al., 2002). These approaches improved representation power but also highlighted sensitivity to data quality, split methodology, and deployment assumptions.

2.4 Deep Learning and Sequence Modeling for HIDS

Recent work frames syscall traces as language-like sequences and applies neural sequence models. LSTM-based system-call language modeling demonstrated strong anomaly discrimination and emphasized the role of thresholding for false alarm control (Kim et al., 2016). Later studies examined representation choices, including abstraction and context enrichment, with measurable trade-offs between recall and false positives (Wunderlich et al., 2020; Lv et al., 2021).

2.5 Datasets and Benchmark Evolution

Dataset quality is a major bottleneck in HIDS research. ADFA-LD and its follow-up evaluations addressed limitations of older corpora but still exposed issues of comparability and representational depth (Creech, 2014; Xie & Hu, 2013; Xie et al., 2014a; Xie et al., 2014b). LID-DS introduced modern scenario design and later expanded metadata richness and tooling support, including thread-level context and standardized processing workflows (Pendleton & Xu, 2017; Grimmer et al., 2019; Grimmer et al., 2022; Grimmer et al., 2023).

2.6 Thresholding, Evaluation, and False Positives

Across studies, high ranking performance does not automatically translate into stable operational decisions. Threshold policy directly shapes false positive rate and analyst workload. Therefore, reportable IDS studies should separate ranking quality (e.g., AUC) from operating-point behavior (precision/recall/FPR/accuracy), and should disclose threshold sensitivity explicitly (Bridges et al., 2019; Talhi, 2017).

2.7 Literature Comparison Table

Table 1 Literature comparison summary for representative HIDS studies.

Study	Dataset	Method	Key Finding	Limitation	Relevance to this project
Forrest et al. (1996)	UNIX traces	Short syscall sequences	Early self/non-self process profiling	Limited modern context	Foundation for sequence anomaly detection
Hofmeyr et al. (1998)	UNM-like syscall data	Sequence-based anomaly detection	Established syscall-sequence IDS paradigm	Older workloads	Baseline theory for HIDS pipelines
Warrender et al. (1999)	System-call benchmark data	STIDE/HMM/RI comparison	Alternative sequence models compared	Dataset age and scope	Motivates classical baselines
Sekar et al. (2001)	Program syscall sequences	Automaton-based detection	Efficient sequence anomaly modeling	Program-specific modeling cost	Efficiency considerations
Tan & Maxion (2002)	STIDE settings	Operational limit analysis	Showed sensitivity to n-gram size	Context-limited setup	Supports careful n choice
Hu et al. (2009)	Host syscall data	HMM-based anomaly detection	Simple efficient HMM scheme	Requires calibration	Statistical sequential baseline

				and feature tuning	
Creech (2014)	ADFA-LD	Semantic system-call patterns	Improved detection on modern attacks	Dataset-specific dependence	Modern benchmark transition
Kim et al. (2016)	Public syscall datasets	LSTM language modeling	Strong deep sequence modeling potential	Threshold tuning needed	Direct LSTM relevance
Grimmer et al. (2022)	LID-DS	Thread-aware extensions	Thread information improves HIDS	Scenario dependence	Supports thread-based construction
Wunderlich et al. (2020)	ADFA-LD & LID-DS	Representation comparison	Representation impacts outcomes	No universal mapping	Supports representation ablation
Park et al. (2021)	LID-DS	Siamese network	Few-shot class-discrimination potential	Different objective	Related LID-DS deep approach
Grimmer et al. (2023)	LID-DS 2021	Dataset/framework report	Improved metadata and tooling	Still synthetic scenarios	Primary dataset reference

2.8 Research Gap

The literature shows strong algorithmic progress, but comparisons are often not directly transferable due to inconsistent split policies, threshold definitions, and trace-level aggregation choices. This capstone addresses that gap at undergraduate scope by implementing a transparent, trace-disjoint, single-scenario comparison on LID-DS 2021 (PHP_CWE-434), with STIDE, LSTM Syscall, and LSTM Context evaluated under aligned trace-level scoring and validation-normal quantile thresholding.

Section 3:

Methodology

3. Methodology

3.1 Research Design

The project follows an experimental comparative design using one classical baseline and two LSTM-based models. The study is an offline trace-level anomaly detection case study, not a real-time production deployment experiment.

3.2 Dataset and Case Description

Dataset: LID-DS 2021. Scenario used in this repository: PHP_CWE-434. Input traces are Linux system-call records processed into labeled windows and aggregated at trace level. Split handling is trace-disjoint, and validation attack traces are separated from test attack traces in pipeline logic and split-check scripts.

3.3 Parsing and Preprocessing Pipeline

Raw .sc traces are parsed into structured records (timestamp, cpu_id, thread_id, process_name, process_id, syscall, direction, result). Cleaning enforces required fields, valid directions, numeric conversion, and derives return_status from syscall result values. Warmup filtering is applied using scenario metadata, then windows are generated per thread.

3.4 Thread-Based Window Construction

Windows are built by grouping events by thread_id, sorting by timestamp, and applying a sliding window of size 30 with stride 1. Windows crossing exploit boundaries are discarded by label assignment logic: windows ending before exploit time are normal, windows starting at/after exploit time are anomaly, and crossing windows are removed.

3.5 Feature Encoding

Two representations are used for LSTM models: (i) syscall-only token sequences from window_syscalls, and (ii) context tokens combining syscall_name, process_name, and return_status triplets. Vocabularies are built from training data with PAD and UNK tokens, and training data encoding is capped at 500,000 windows.

3.6 Models and Training Configuration

STIDE uses n-gram size n=6 and mismatch ratio scoring. LSTM models use next-token prediction with embedding dimension 64, hidden dimension 128, one LSTM layer, dropout 0.2, batch size 256, Adam optimizer (lr=1e-3), CrossEntropyLoss with PAD ignored, and 10 epochs.

Training uses normal windows only (only_normal=True), while evaluation includes normal and attack traces.

3.7 Trace-Level Aggregation and Thresholding

Window anomaly scores are aggregated to trace-level scores via top-10% mean pooling. For a trace with window scores s_(1) >= s_(2) >= ... >= s_(n), k = ceil(0.10*n), and trace_score = (1/k) * sum_{i=1..k} s_(i). Primary operating thresholds are validation-normal q95 values; q90 is used as sensitivity analysis. Original validation-based thresholding is retained only as supporting comparison.

3.8 Evaluation Metrics

Precision = TP/(TP+FP), Recall = TP/(TP+FN), F1 = 2*(Precision*Recall)/(Precision+Recall), FPR = FP/(FP+TN), Accuracy = (TP+TN)/(TP+TN+FP+FN). ROC-AUC is reported as threshold-independent ranking quality.

3.9 Implementation Environment and Repository Structure

Implementation is Python-based with modular source folders: parsing, preprocessing, pipeline, baseline, lstm, reporting, and demo. Final report figures and tables are generated from dedicated reporting scripts and exported under results/final_figures and results/final_tables.

3.10 Dataset and Experimental Setup Summary

Table 2 Dataset and experimental setup used in this project.

Item	Configured value
Dataset	LID-DS 2021
Scenario	PHP_CWE-434
Detection setting	Offline trace-level anomaly detection prototype
Input signals	Linux system call traces
Core fields	syscall_name, process_name, return_status, thread_id
Window generation	thread_id grouping, window=30, stride=1
Boundary rule	Exploit-crossing windows discarded
Models	STIDE (n=6), LSTM Syscall, LSTM Context
Training policy	Normal windows only
Train cap	500,000 windows for LSTM training/encoding
LSTM config	emb=64, hidden=128, layers=1, dropout=0.2
Optimizer/loss	Adam lr=1e-3, CrossEntropyLoss(ignore PAD)

Batch/Epoch	256 / 10
Trace aggregation	Top-10% mean pooling
Main threshold	Validation-normal q95
Sensitivity threshold	Validation-normal q90

Section 4:

Results

4. Results

4.1 Experimental Setup and Operating Points

The main operating point is validation-normal q95 thresholding for all three models. Fixed q95 thresholds are STIDE=0.145818, LSTM Syscall=0.895200, and LSTM Context=1.067240. Sensitivity analysis uses q90 thresholds: STIDE=0.054867, LSTM Syscall=0.575347, and LSTM Context=0.915675.

4.2 ROC-AUC Summary

Table 3 ROC-AUC comparison on test traces.

model	auc_roc
STIDE	0.9947
LSTM Syscall	0.9962
LSTM Context	0.9915

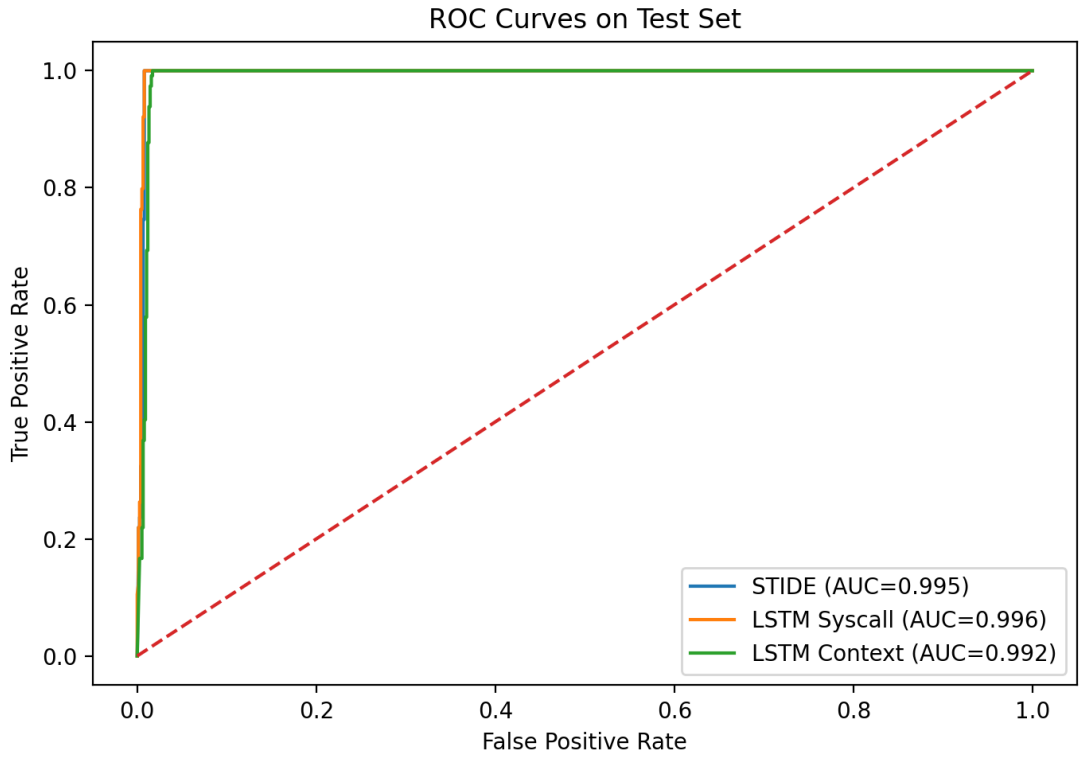

Figure 1 Combined ROC curves for STIDE, LSTM Syscall, and LSTM Context.

All three models achieve AUC above 0.99 on this test set, indicating strong ranking ability. However, operating-point metrics still differ materially across threshold policies.

4.3 Main Comparison at Validation-Normal q95

Table 4 Main trace-level comparison at validation-normal q95 thresholds.

model	threshold	precision	recall	f1	fpr	accuracy	tp	tn	fp	fn
STIDE	0.1458	0.9412	0.8421	0.8889	0.0079	0.9726	96	756	6	18
LSTM Syscall	0.8952	0.9469	0.9386	0.9427	0.0079	0.9852	107	756	6	7
LSTM Context	1.0672	0.8976	1.0000	0.9461	0.0171	0.9852	114	749	13	0

At q95, LSTM Syscall yields the strongest balance: it substantially improves recall over STIDE while preserving the same low FPR (0.0079). LSTM Context reaches perfect recall (1.0000) but with higher false positives (FPR 0.0171). STIDE remains a strong and interpretable baseline.

4.4 Confusion Matrix and Score Distribution Analysis

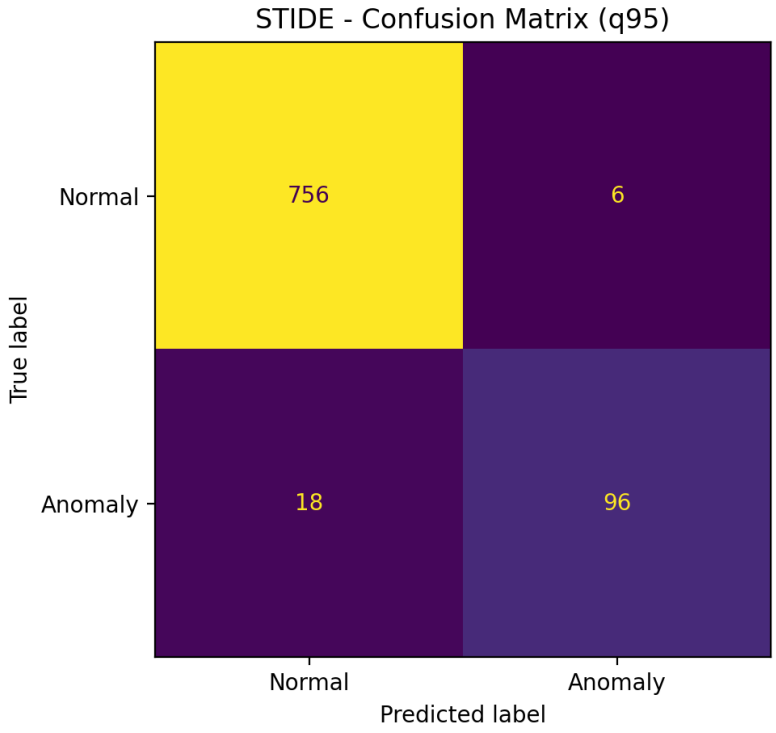

Figure 2 STIDE confusion matrix at q95.

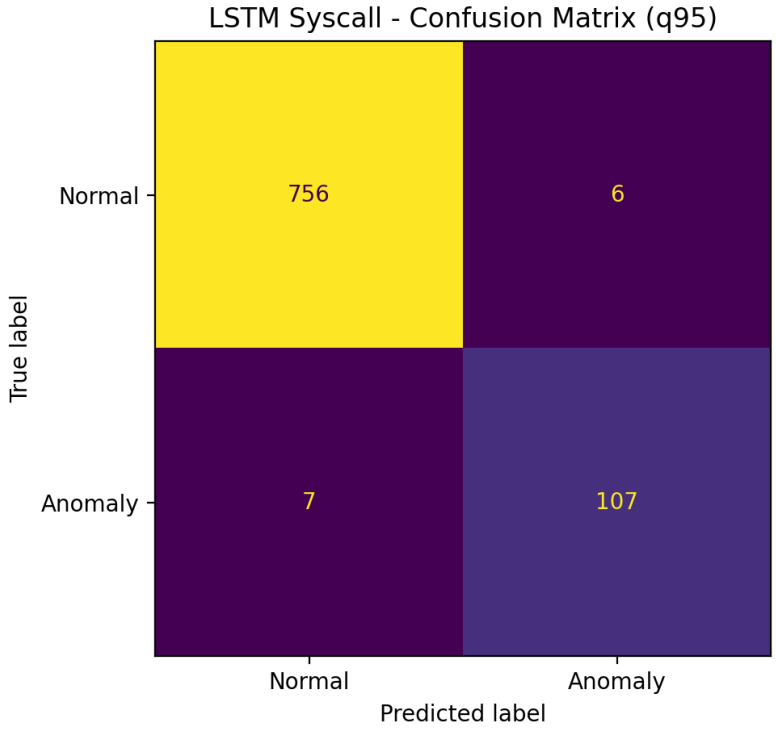

Figure 3 LSTM Syscall confusion matrix at q95.

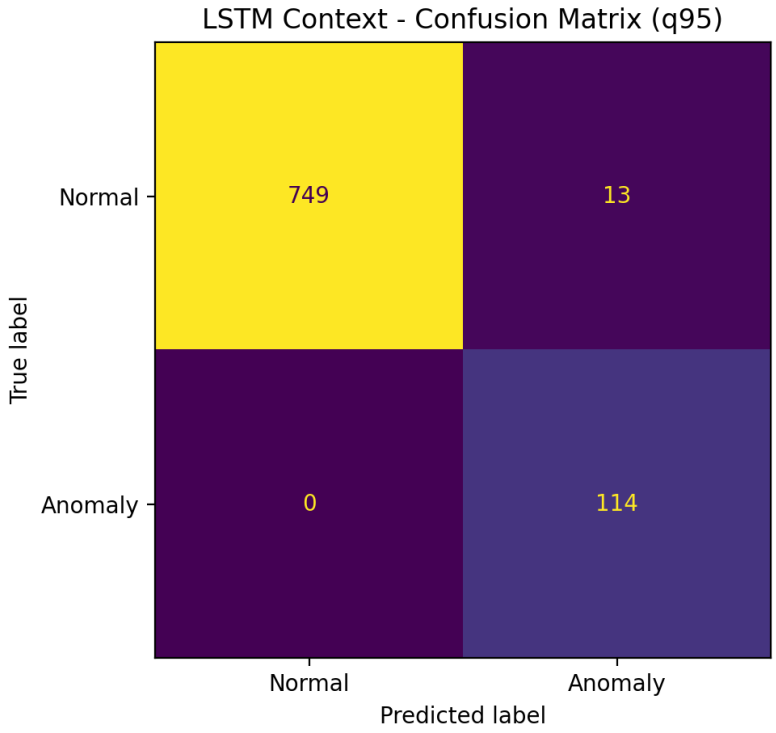

Figure 4 LSTM Context confusion matrix at q95.

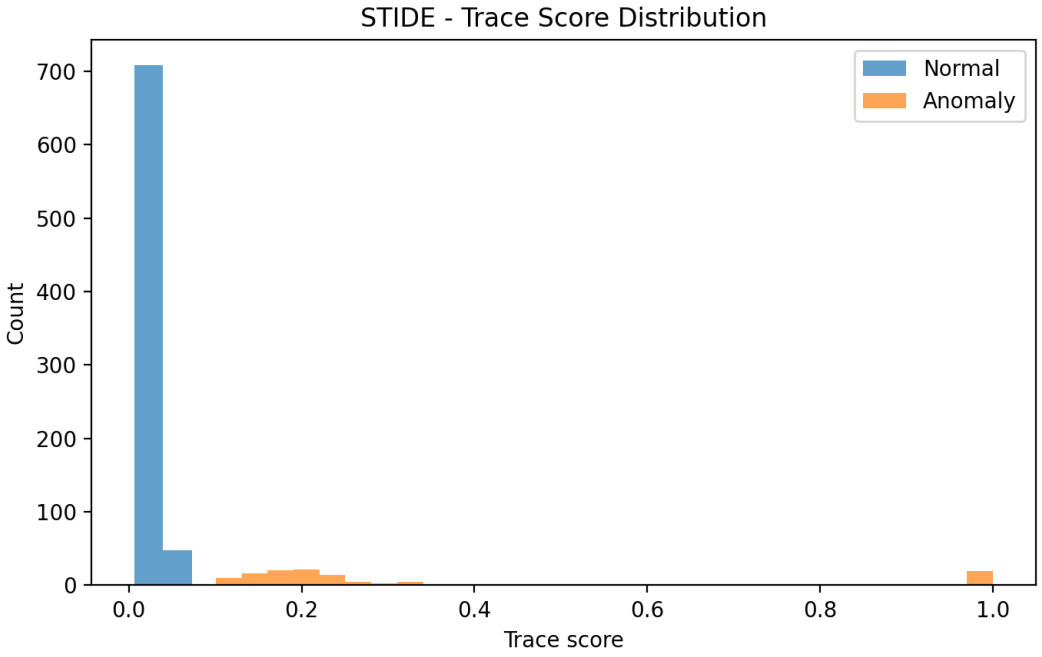

Figure 5 STIDE trace-score distribution.

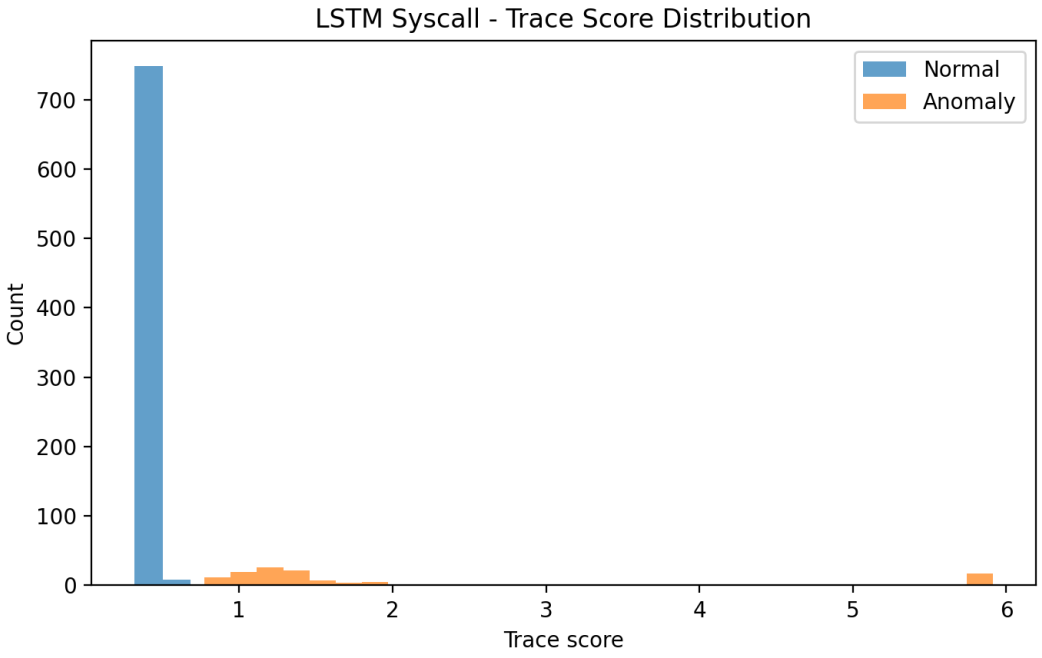

Figure 6 LSTM Syscall trace-score distribution.

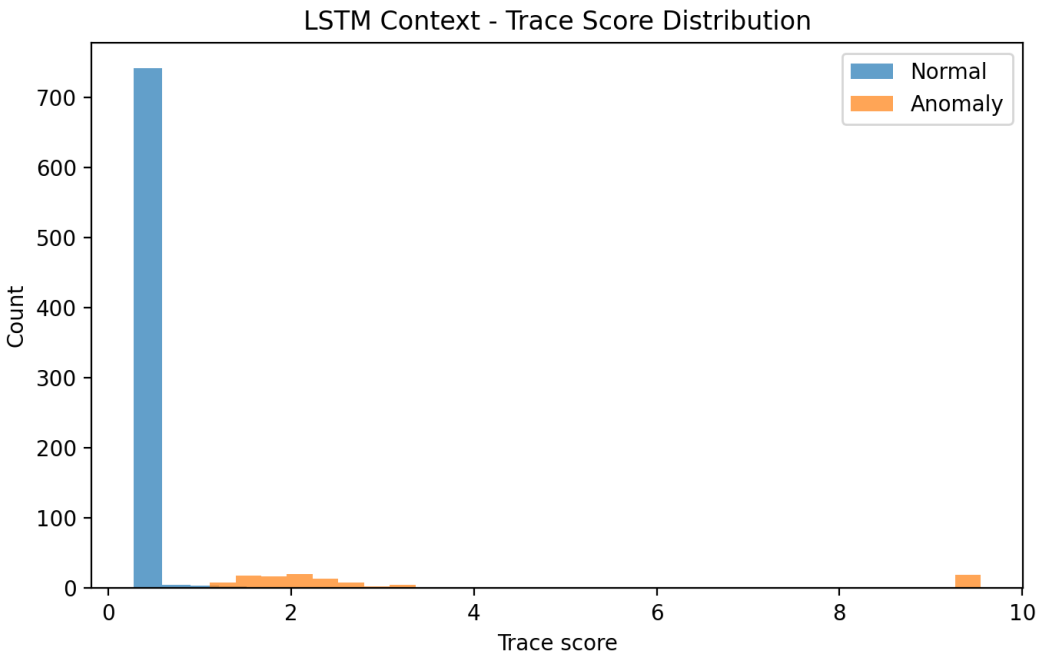

Figure 7 LSTM Context trace-score distribution.

Confusion matrices and score histograms confirm the threshold-level trade-off: context-aware modeling catches all attack traces at q95 but shifts part of normal traffic to false positives. Syscall-only LSTM shows tighter separation under the selected operating point.

4.5 Sensitivity Analysis with q90 Thresholds

Table 5 Sensitivity comparison at validation-normal q90 thresholds.

model	threshold	precision	recall	f1	fpr	accuracy	tp	tn	fp	fn
STIDE	0.0549	0.9194	1.0000	0.9580	0.0131	0.9886	114	752	10	0
LSTM Syscall	0.5753	0.9421	1.0000	0.9702	0.0092	0.9920	114	755	7	0
LSTM Context	0.9157	0.8769	1.0000	0.9344	0.0210	0.9817	114	746	16	0

At q90, recall becomes 1.0000 for all three models, but FPR rises in each case. This confirms that threshold policy significantly affects operational burden even when ranking performance is high.

4.6 Original Thresholding and Reliability Summaries

Table 6 Original validation-based thresholding comparison (supporting).

model	threshold_strategy	precision	recall	f1	fpr	accuracy
STIDE	original validation-based	0.9455	0.9123	0.9286	0.0079	0.9817
LSTM Syscall	original validation-based	0.9375	0.2632	0.4110	0.0026	0.9018
LSTM Context	original validation-based	0.9174	0.8772	0.8969	0.0118	0.9737

Table 7 Bootstrap q95 reliability summary (mean metrics).

model	threshold_mean	precision_mean	recall_mean	f1_mean	fpr_mean	accuracy_mean
STIDE	0.1528	0.9180	0.7230	0.7562	0.0093	0.9558
LSTM Syscall	0.9458	0.9338	0.7597	0.7941	0.0076	0.9621
LSTM Context	1.1005	0.8923	0.9746	0.9307	0.0177	0.9813

Table 8 Error analysis summary at q95.

model	threshold	fp_count	fn_count	tp_count	tn_count
STIDE	0.1458	6	18	96	756
LSTM Syscall	0.8952	6	7	107	756
LSTM Context	1.0672	13	0	114	749

Original validation-based thresholding can underperform for specific models (notably LSTM Syscall in this repository output), which motivates validation-normal quantile calibration for fairer cross-model comparison. Bootstrap summaries indicate generally stable q95 behavior but

wider confidence intervals for recall-focused metrics, consistent with limited anomalous trace counts.

4.7 Summary of Findings

The central empirical finding is that LSTM Syscall is the best balanced model for this case study, LSTM Context is recall-dominant but less conservative in false positives, and STIDE remains unexpectedly strong. Therefore, classical baselines should be retained in modern HIDS benchmarking rather than treated as obsolete.

Section 5:

Discussion and Conclusion

5. Discussion and Conslusion

5.1 Discussion of Key Findings

The results support the research question in this single-scenario setting: anomalous process behavior can be detected from normal syscall patterns with high ranking quality and strong trace-level metrics. However, the practical trade-off between recall and false positives depends on both representation choice and threshold calibration.

5.2 Practical Implications

For security operations, the most useful outcome is not only model ranking, but calibration-aware deployment choice. In this project scope, LSTM Syscall at q95 offers a practical balance, while LSTM Context may be preferable when missing attacks is costlier than additional triage workload.

5.3 Limitations

Key limitations are: single-scenario focus (PHP_CWE-434), offline trace-level prototype, no live streaming deployment, capped training subset (500,000 windows), and limited validation attack diversity. As a result, generalization claims to broader enterprise environments would be inappropriate.

5.4 Future Work

Future work should evaluate multiple LID-DS scenarios and external datasets (e.g., ADFA-LD), test additional architectures (GRU/BiLSTM/Transformer/autoencoder-style), improve explainability and threshold calibration, and assess real-time streaming feasibility with reproducible containerized workflows.

5.5 Conclusion

This capstone delivered a reproducible comparative HIDS pipeline and showed that, in the studied scenario, LSTM Syscall gives the strongest precision-recall-FPR balance under validation-normal q95 calibration. The study also demonstrated that strong AUC alone is insufficient for operational decisions and that transparent threshold policy reporting is essential for honest IDS evaluation.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor for guidance and feedback throughout this capstone project. I also thank the Department of Computer Engineering at Istinye University for providing the academic framework for this study.

APPENDIX

Appendix A. Supporting materials include additional thresholding outputs, profile plots for representative TP/TN/FP/FN traces, and implementation notes for the local demo workflow.

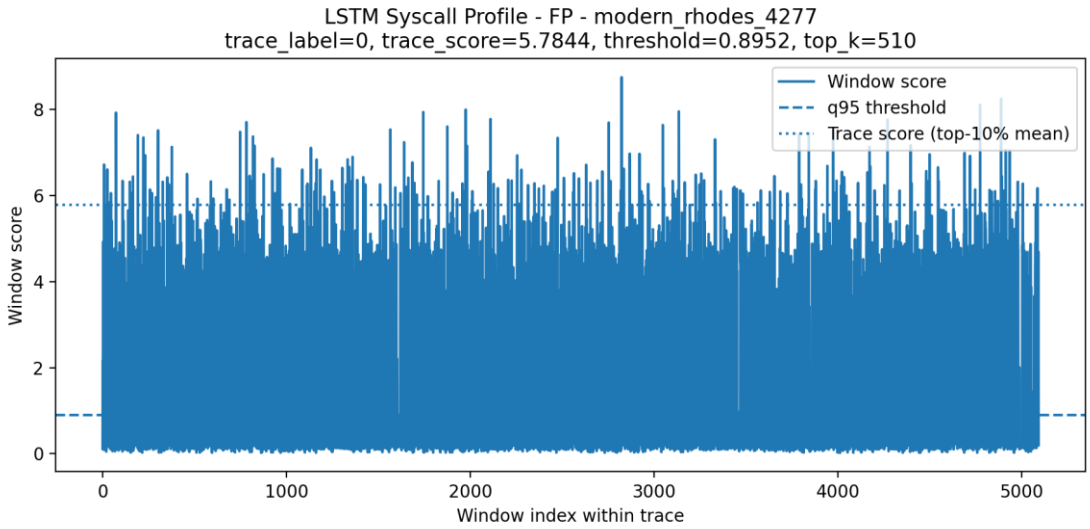

Figure 8 Example false-positive trace profile (q95, LSTM Syscall).

The source code, preprocessing scripts, trained model outputs, additional figures, and demo workflow are available in the project GitHub repository:

GitHub Repository: https://github.com/Emirhanmu/LSTM-Based-HIDS-on-System-Call-Sequences

REFERENCES

References

Bridges, R. A., Glass-Vanderlan, T. R., Iannacone, M. D., Vincent, M. S., & Chen, Q. (2019). A survey of intrusion detection systems leveraging host data. ACM Computing Surveys, 52(6), Article 128. https://doi.org/10.1145/3344382

Creech, G. (2014). Developing a high-accuracy cross platform host-based intrusion detection system capable of reliably detecting zero-day attacks (PhD thesis). UNSW Sydney.

Forrest, S., Hofmeyr, S. A., Somayaji, A., & Longstaff, T. A. (1996). A sense of self for Unix processes. In Proceedings of the 1996 IEEE Symposium on Security and Privacy (pp. 120-128).

Grimmer, M., Kaelble, T., & Rahm, E. (2022). Improving host-based intrusion detection using thread information. In EISA 2021 (CCIS 1403, pp. 159-177). Springer. https://doi.org/10.1007/978-3-030-93956-4_10

Grimmer, M., Rohling, M. M., Kreusel, D., & Ganz, S. (2019). A modern and sophisticated host based intrusion detection data set. 16. Deutscher IT-Sicherheitskongress.

Grimmer, M., Nirsberger, F., Kaelble, T., Schulze, E., Rucks, T., Rohling, M. M., Kreusel, D., & Ganz, S. (2023). Dataset report: LID-DS 2021. In Detection of Intrusions and Malware, and Vulnerability Assessment (pp. 59-74). Springer. https://doi.org/10.1007/978-3-031-35190-7_6

Hofmeyr, S. A., Forrest, S., & Somayaji, A. (1998). Intrusion detection using sequences of system calls. Journal of Computer Security, 6(3), 151-180. https://doi.org/10.3233/JCS-980109

Hoang, X. D., Hu, J., & Bertok, P. (2009). A program-based anomaly intrusion detection scheme using multiple detection engines and fuzzy inference. Journal of Network and Computer Applications, 32(6), 1219-1228. https://doi.org/10.1016/j.jnca.2009.05.004

Hu, J., Yu, X., Qiu, D., & Chen, H.-H. (2009). A simple and efficient hidden Markov model scheme for host-based anomaly intrusion detection. IEEE Network, 23(1), 42-47. https://doi.org/10.1109/MNET.2009.4804323

Kim, G., Yi, H., Lee, J., Paek, Y., & Yoon, S. (2016). LSTM-based system-call language modeling and robust ensemble method for designing host-based intrusion detection systems. arXiv. https://doi.org/10.48550/arXiv.1611.01726

Liao, Y., & Vemuri, V. R. (2003). Host-based intrusion detection using dynamic and static behavioral models. Pattern Recognition, 36(1), 229-243. https://doi.org/10.1016/S0031-3203(02)00026-2

Lv, Y., Wang, C., Huang, M., Liang, H., & Yu, H. (2021). Syscall-BSEM: Behavioral semantics enhancement method of system call sequence for high accurate and robust host intrusion detection. Future Generation Computer Systems, 126, 201-214. https://doi.org/10.1016/j.future.2021.06.030

Park, D., Kim, S., Kwon, H., Shin, D., & Shin, D. (2021). Host-based intrusion detection model using Siamese network. IEEE Access, 9, 76614-76623. https://doi.org/10.1109/ACCESS.2021.3082160
Pendleton, M., & Xu, S. (2017). A dataset generator for next generation system call host intrusion detection systems. In MILCOM 2017 (pp. 231-236). https://doi.org/10.1109/MILCOM.2017.8170835
Rohling, M. M., Grimmer, M., Kreubel, D., Hoffmann, J., & Franczyk, B. (2019). Standardized container virtualization approach for collecting host intrusion detection data. In FedCSIS 2019 (pp. 459-463).
Sekar, R., Bendre, M., Dhurjati, D., & Bollineni, P. (2001). A fast automaton-based method for detecting anomalous program behaviors. In Proceedings of the 2001 IEEE Symposium on Security and Privacy (pp. 144-155). https://doi.org/10.1109/SECPRI.2001.924295
Talhi, C. (2017). An anomaly detection system based on variable N-gram features and one-class SVM. Information and Software Technology, 91, 186-197. https://doi.org/10.1016/j.infsof.2017.07.009
Tan, K. M., & Maxion, R. A. (2002). Why 6? Defining the operational limits of STIDE, an anomaly-based intrusion detector. In Proceedings of the 2002 IEEE Symposium on Security and Privacy (pp. 188-201).
Tandon, G., & Chan, P. K. (2006). On the learning of system call attributes for host-based anomaly detection. International Journal on Artificial Intelligence Tools, 15(6), 875-892.
Wagner, D., & Dean, D. (2001). Intrusion detection via static analysis. In Proceedings of the 2001 IEEE Symposium on Security and Privacy (pp. 156-168). https://doi.org/10.1109/SECPRI.2001.924296
Wagner, D., & Soto, P. (2002). Mimicry attacks on host-based intrusion detection systems. In Proceedings of the 9th ACM Conference on Computer and Communications Security (pp. 255-264). https://doi.org/10.1145/586110.586145
Warrender, C., Forrest, S., & Pearlmutter, B. (1999). Detecting intrusions using system calls: Alternative data models. In Proceedings of the 1999 IEEE Symposium on Security and Privacy (pp. 133-145). https://doi.org/10.1109/SECPRI.1999.766910
Wunderlich, S., Ring, M., Landes, D., & Hotho, A. (2020). Comparison of system call representations for intrusion detection. In CISIS/ICEUTE 2019 (AISC 951, pp. 14-24). Springer. https://doi.org/10.1007/978-3-030-20005-3_2
Xie, M., & Hu, J. (2013). Evaluating host-based anomaly detection systems: A preliminary analysis of ADFA-LD. In 6th IEEE International Congress on Image and Signal Processing (pp. 1711-1716). https://doi.org/10.1109/CISP.2013.6743952
Xie, M., Hu, J., & Slay, J. (2014a). Evaluating host-based anomaly detection systems: Application of the one-class SVM algorithm to ADFA-LD. In 11th International Conference on Fuzzy Systems and Knowledge Discovery (pp. 978-982).

Xie, M., Hu, J., Yu, X., & Chang, E. (2014b). Evaluating host-based anomaly detection systems: Application of the frequency-based algorithms to ADFA-LD. In International Conference on Network and System Security (pp. 542-549). Springer.
Ye, N., Li, X., Chen, Q., Emran, S. M., & Xu, M. (2001). Probabilistic techniques for intrusion detection based on computer audit data. IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans, 31(4), 266-274.
Ye, N., Emran, S. M., Chen, Q., & Vilbert, S. (2002). Multivariate statistical analysis of audit trails for host-based intrusion detection. IEEE Transactions on Computers, 51(7), 810-820.